Лабораторна робота №9

Тонке налаштування нейронної мережі (Fine Tuning)

Виконав: студент групи 443-4 **Бабин Богдан Олегович**

Мета роботи

Навчитися технології тонкого налаштування (fine tuning) попередньо навченої мережі в Keras: після перенесення навчання розморозити останні згорткові блоки і донавчити їх на своїх даних. Підібрати гіперпараметри класифікатора, порівняти VGG16, InceptionV3 і ResNet50 з результатами лабораторної 8.

Посилання

- 9.1 VGG16: https://colab.research.google.com/drive/12okYtQR7_z2Qvvnct7-PyF1tEYq_GiCL?usp=sharing
- 9.2 InceptionV3: https://colab.research.google.com/drive/1jD8Y4X9_CqomD443Y4clUurMEgkZn3Lj?usp=sharing
- 9.3 ResNet50: https://colab.research.google.com/drive/1jbmByFk7f4SZHclxxojXpOD3DwUELWmJ?usp=sharing
- Папка на Google Drive (ноутбуки з результатами): https://drive.google.com/drive/folders/1YMXxqPfgJsesW6Hx2PwfE8MpbACRNOcz?usp=sharing

1. Що таке тонке налаштування

У лабораторній 8 згорткова частина була повністю **заморожена** — мережа використовувала ознаки, вивчені на ImageNet, без змін. Перші шари виділяють загальні ознаки (краї, кольори, текстури), і вони підходять для будь-яких фото. Останні блоки виділяють складніші ознаки, налаштовані під 1000 класів ImageNet. **Тонке налаштування** — це донавчання саме цих останніх блоків разом із класифікатором, щоб ознаки краще підходили для «кіт / собака». Порядок:

- спочатку перенесення навчання: навчаємо лише класифікатор (інакше великі градієнти від випадково ініціалізованого класифікатора зруйнують ваги ImageNet);
- розморожуємо 1–2 останні блоки (`layer.trainable = True`), шари BatchNormalization залишаємо замороженими;
- перекомпілюємо модель і навчаємо з **дуже малою** швидкістю (1e-5), щоб ваги змінювались лише трохи.

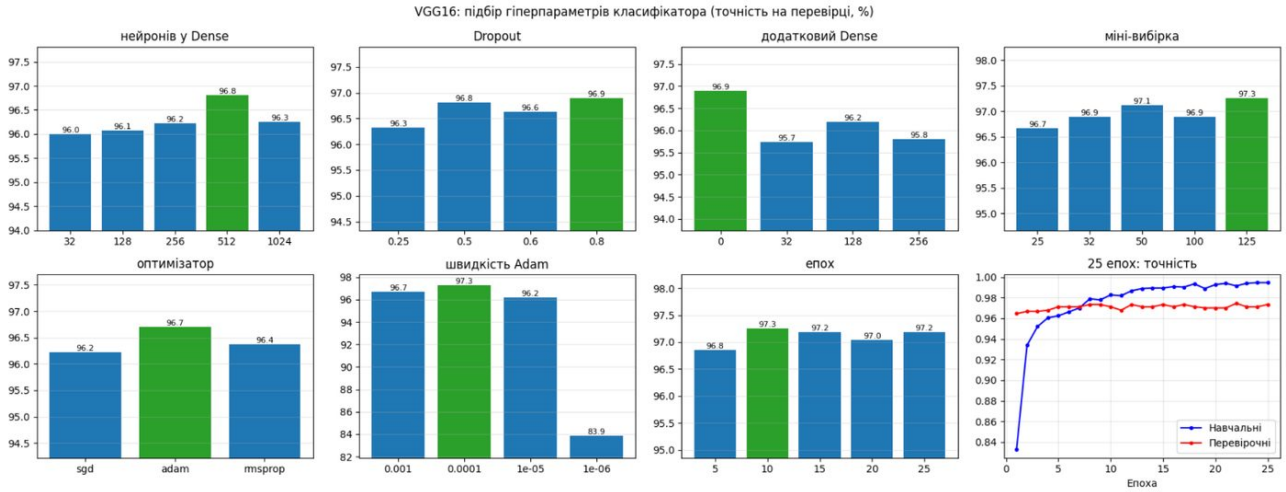
```
base_net.trainable = True
cut = names.index('block4_pool')                  # 1 блок = усе після block4_pool, тобто block5
for i, layer in enumerate(base_net.layers):
    layer.trainable = i > cut and not isinstance(layer, BatchNormalization)
model.compile(loss='binary_crossentropy', optimizer=Adam(learning_rate=1e-5), metrics=['accuracy'])
model.fit(train_generator, epochs=10, validation_data=val_generator,
          callbacks=[EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)])
```

2. Як виконано

- **Дані** — той самий скрипт з лаб 7–8: Dogs vs Cats, по 3 000 фото кожного класу, train 4 200 / val 900 / test 900 (щоб результати можна було порівняти з лабою 8).
- **Підбір гіперпараметрів** (усі пункти Exercises): нейронів у Dense 32–1024, Dropout 0,25–0,8, додатковий Dense 32/128/256, міні-вибірка 25–125, оптимізатори sgd/adam/rmsprop, швидкість Adam 1e-3…1e-6, епох 5–25. Поки згорткова частина заморожена, її вихід для кожного фото не змінюється, тому ознаки обчислено **один раз**, а класифікатор навчався на них — 30 експериментів за ~4 хв. Пошук жадібний: змінюємо один параметр, найкраще значення залишаємо для наступних. Вибір — за точністю на **перевірочних** даних (середнє за 3 останні епохи), тест для вибору не використовується.
- **Тонке налаштування**: 4 варіанти — 1 блок з lr 1e-5, 1e-4, 1e-6 і 2 блоки з lr 1e-5. Кожен починається з тих самих ваг після перенесення навчання.
- Найкраща модель кожної мережі збережена (`lab9_<мережа>_best.keras`) і перевірена на власних фото.

3. VGG16

Перенесення навчання: підбір гіперпараметрів

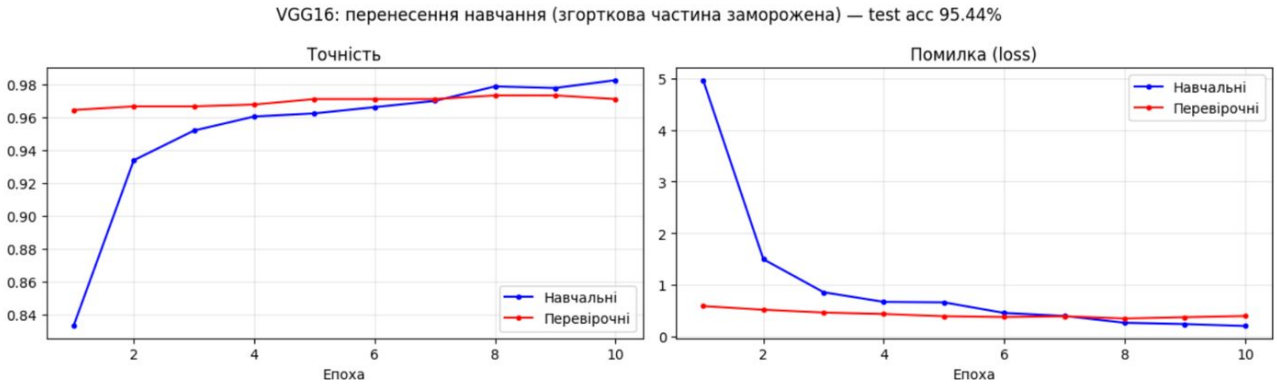


VGG16: точність на перевірці для кожного варіанта (зелене — обране значення)

Гіперпараметр	Значення → точність на перевірці, %	Обрано
Нейронів у Dense	32: 96,00, 128: 96,07, 256: 96,22, **512: 96,81**, 1024: 96,26	512
Dropout	0.25: 96,33, 0.5: 96,81, 0.6: 96,63, **0.8: 96,89**	0.8
Додатковий Dense	**немає: 96,89**, 32: 95,74, 128: 96,19, 256: 95,81	немає
Міні-вибірка	25: 96,67, 32: 96,89, 50: 97,11, 100: 96,89, **125: 97,26**	125
Оптимізатор	sgd: 96,22, **adam: 96,70**, rmsprop: 96,37	adam
Швидкість Adam	1e-3: 96,70, **1e-4: 97,26**, 1e-5: 96,19, 1e-6: 83,89	1e-4
Епох	5: 96,85, **10: 97,26**, 15: 97,19, 20: 97,04, 25: 97,19	10

Найбільше впливають швидкість навчання і розмір класифікатора. Після Flatten у VGG16 8 192 входи, тому Dense 512 має 4,2 млн параметрів — і найкраще працює з сильним Dropout 0,8. Додатковий Dense лише погіршує (95,7–96,2%): більше параметрів — більше перенавчання. Більша міні-вибірка (125) дала трохи кращий результат — кроки градієнта

менш «шумні». Серед оптимізаторів зі стандартною швидкістю найкращий Adam; RMSprop має найбільші втрати на перевірці (0,61) — модель занадто впевнена. Adam 1e-4 — оптимум: 1e-5 за 10 епох не встигає навчитися (train 89,6%), а 1e-6 — майже не вчиться (83,9%). Після 10 епох точність на перевірці вже не росте, а на навчанні доходить до 99,5% — далі лише перенавчання. Обрано: **Dense 512, Dropout 0,8, batch 125, Adam 1e-4, 10 епох**.

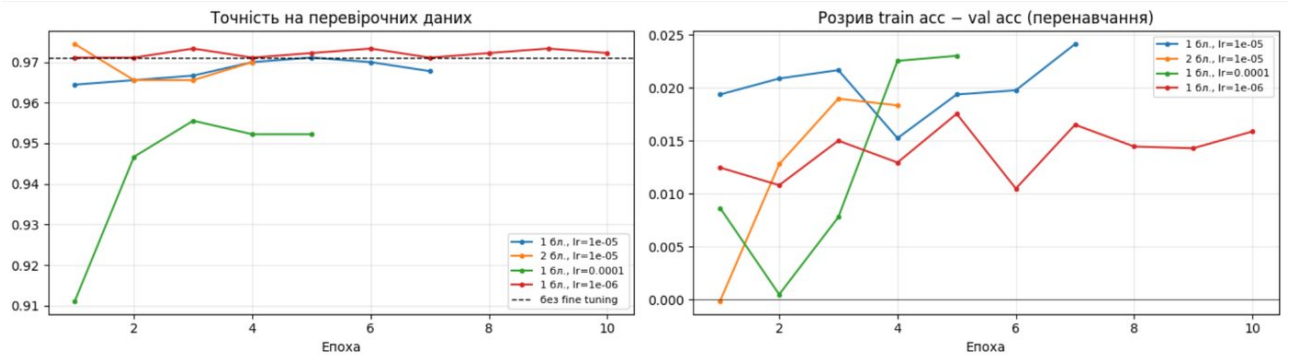


VGG16: перенесення навчання з підібраними гіперпараметрами — test acc 95,44% (лаба 8: 97,56%)

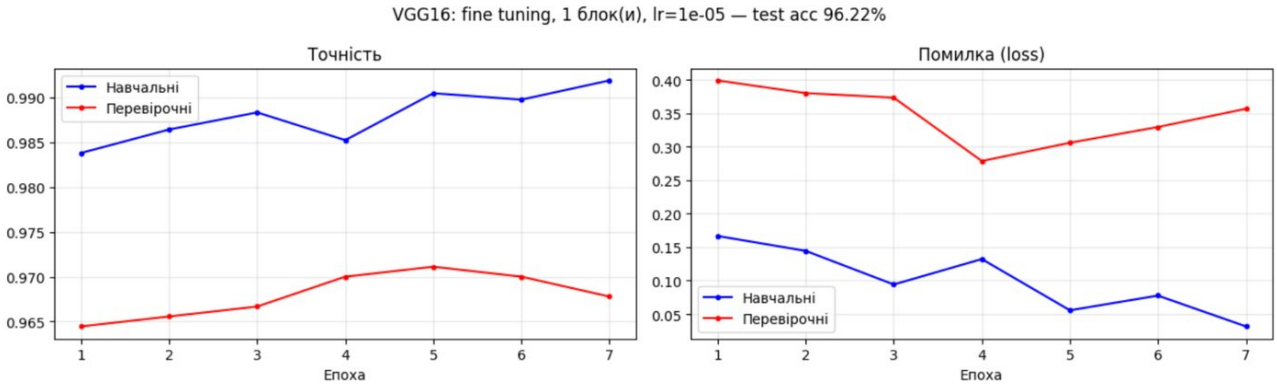
Тонке налаштування

Розморожено 1 блок (block5) або 2 блоки (block4 + block5); BatchNormalization заморожені; Adam з різною швидкістю, до 10 епох з EarlyStopping за val loss (patience 3).

Блоків	lr	Навчаються параметрів	Епох (найкр.)	Train acc	Val acc	Val loss	Test acc, %	Test loss	Час, с
1	1e-5	11 274 753	7 (4)	98,52	97,00	0,2787	**96,22**	0,4952	126
2	1e-5	17 174 529	4 (1)	97,43	97,44	0,1963	94,78	0,5073	99
1	1e-4	11 274 753	5 (2)	94,71	94,67	0,1728	93,67	0,1888	89
1	1e-6	11 274 753	10 (9)	98,76	97,33	0,3586	95,89	0,6384	169



VGG16: точність на перевірці і розрив train − val для варіантів fine tuning (пунктир — без fine tuning)

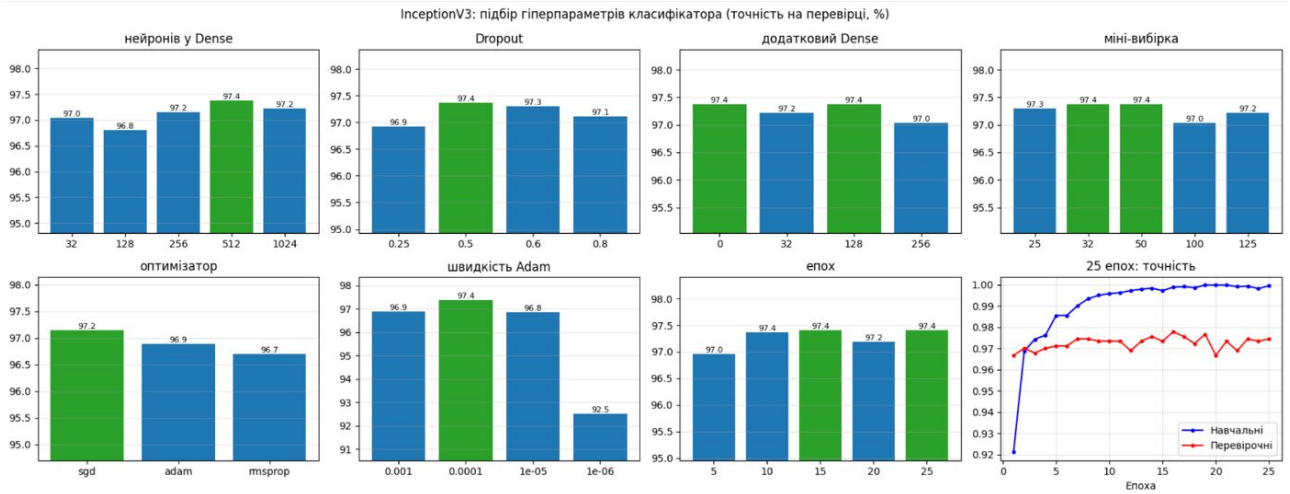


VGG16: fine tuning, 1 блок, lr=1e-5 — test acc 96,22%

На тесті модель з підібраними гіперпараметрами дала 95,44% — менше, ніж у лабі 8 (97,56%). Вибір робився за перевірочними даними, а різниця між варіантами там 0,3–0,5%, тобто 3–5 фото з 900 — це в межах випадкового розкиду, тому «найкращий на перевірці» не обов'язково найкращий на тесті. Тонке налаштування **1 блоку з lr 1e-5** покращило результат до **96,22%** (+0,8%) і зменшило втрати на перевірці (0,40 → 0,28 на 4-й епосі). 2 блоки (17 млн параметрів) перенавчаються вже після 1-ї епохи (94,78%). Швидкість 1e-4 занадто велика: на першій епосі точність на перевірці падає до 91% — ваги ImageNet «ламаються» (93,67%). Швидкість 1e-6 занадто мала — ваги майже не змінюються (95,89%).

4. InceptionV3

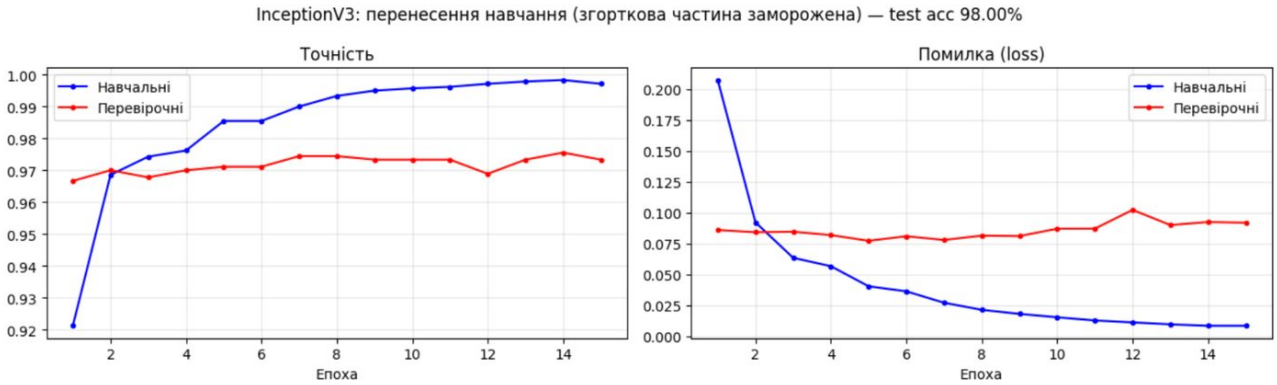
Перенесення навчання: підбір гіперпараметрів



InceptionV3: точність на перевірці для кожного варіанта (зелене — обране значення)

Гіперпараметр	Значення → точність на перевірці, %	Обрано
Нейронів у Dense	32: 97,04, 128: 96,81, 256: 97,15, **512: 97,37**, 1024: 97,22	512
Dropout	0.25: 96,93, **0.5: 97,37**, 0.6: 97,30, 0.8: 97,11	0.5
Додатковий Dense	**немає: 97,37**, 32: 97,22, **128: 97,37**, 256: 97,04	немає
Міні-вибірка	25: 97,30, **32: 97,37**, **50: 97,37**, 100: 97,04, 125: 97,22	32
Оптимізатор	**sgd: 97,15**, adam: 96,89, rmsprop: 96,70	sgd
Швидкість Adam	1e-3: 96,89, **1e-4: 97,37**, 1e-5: 96,85, 1e-6: 92,52	1e-4
Епох	5: 96,96, 10: 97,37, **15: 97,41**, 20: 97,19, **25: 97,41**	15

Для InceptionV3 усі варіанти дуже близькі (96,8–97,4%): після GlobalAveragePooling ознак лише 2 048, і класифікатор маленький. Найкраще — Dense 512, Dropout 0,5, без додаткового шару, batch 32. Серед оптимізаторів зі стандартною швидкістю найкращий SGD (0,01): для Adam і RMSprop стандартна швидкість 0,001 завелика (RMSprop — втрати 0,24). Тому з Adam 1e-4 (97,37%) результат кращий, ніж з SGD, і обрано саме його. 1e-6 знову занадто мала швидкість (92,5%). Найкраща кількість епох — 15 (25 дає те саме). Обрано: **Dense 512, Dropout 0,5, batch 32, Adam 1e-4, 15 епох** — на тесті **98,00%** (лаба 8: 96,89%, +1,1%).

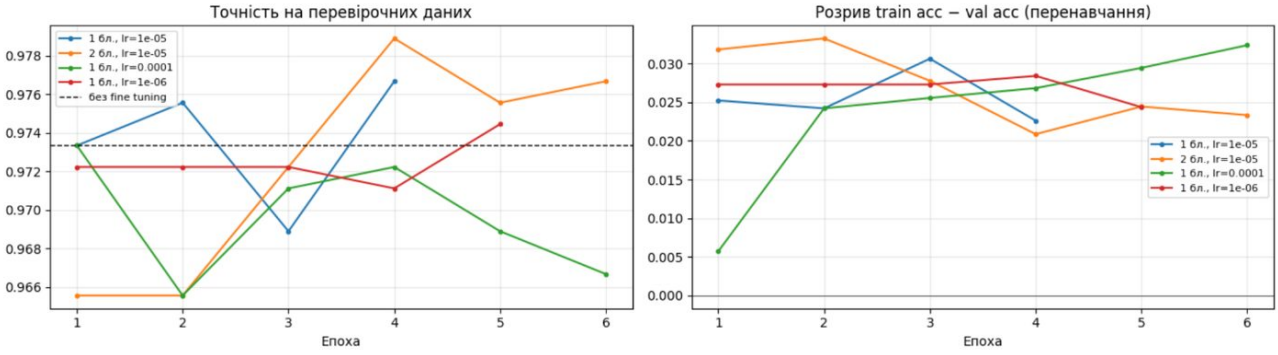


InceptionV3: перенесення навчання з підібраними гіперпараметрами — test acc 98,00% (лаба 8: 96,89%)

Тонке налаштування

Розморожено 1 блок (mixed10) або 2 блоки (mixed9 + mixed10); BatchNormalization заморожені; Adam з різною швидкістю, до 10 епох з EarlyStopping за val loss (patience 3).

Блоків	lr	Навчаються параметрів	Епох (найкр.)	Train acc	Val acc	Val loss	Test acc, %	Test loss	Час, с
1	1e-5	7 119 873	4 (1)	99,86	97,33	0,1002	97,89	0,0759	87
2	1e-5	12 157 953	6 (3)	100,00	97,22	0,1097	**98,00**	0,0851	120
1	1e-4	7 119 873	6 (3)	99,67	97,11	0,0903	97,67	0,0723	106
1	1e-6	7 119 873	5 (2)	99,95	97,22	0,0930	**98,00**	0,0650	93



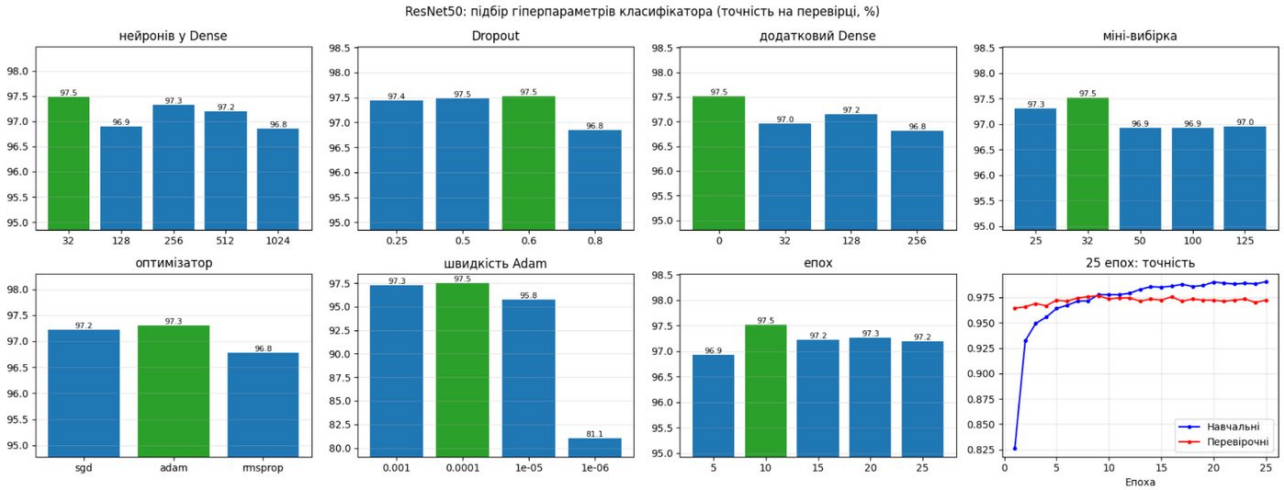
InceptionV3: точність на перевірці і розрив train − val для варіантів fine tuning (пунктир — без fine tuning)

Тонке налаштування InceptionV3 точність не підвищило: 97,67–98,00% проти 98,00% без нього. Мережа вже на 99,9–100% вивчила навчальні фото, тому донавчання розморожених блоків лише збільшує перенавчання — найкраща епоха 1–3, після чого втрати на перевірці ростуть і EarlyStopping зупиняє навчання на 4–6 епосі. Найкращим лишилось перенесення навчання з підібраними гіперпараметрами; варіант з lr 1e-6 дав ту саму точність 98,00% і

найменші втрати на тесті (0,065).

5. ResNet50

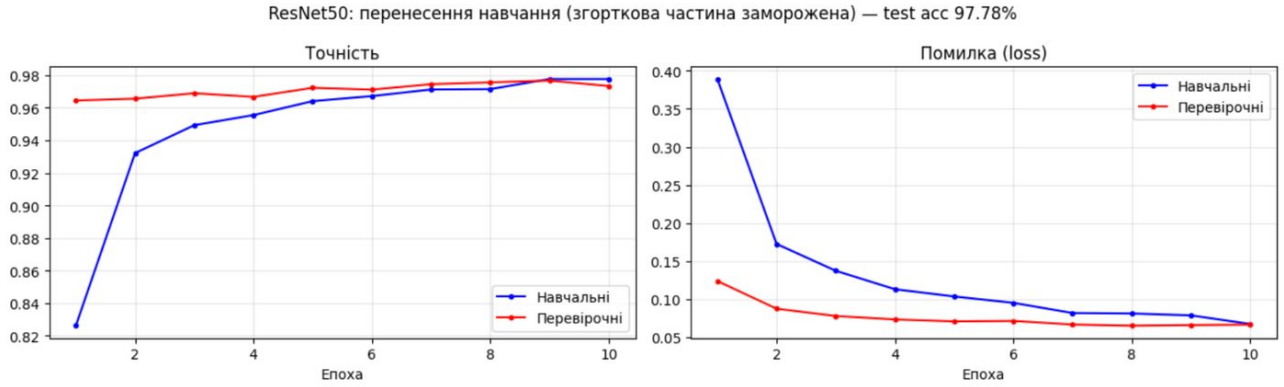
Перенесення навчання: підбір гіперпараметрів



ResNet50: точність на перевірці для кожного варіанта (зелене — обране значення)

Гіперпараметр	Значення → точність на перевірці, %	Обрано
Нейронів у Dense	**32: 97,48**, 128: 96,89, 256: 97,33, 512: 97,19, 1024: 96,85	32
Dropout	0.25: 97,44, 0.5: 97,48, **0.6: 97,52**, 0.8: 96,85	0.6
Додатковий Dense	**немає: 97,52**, 32: 96,96, 128: 97,15, 256: 96,81	немає
Міні-вибірка	25: 97,30, **32: 97,52**, 50: 96,93, 100: 96,93, 125: 96,96	32
Оптимізатор	sgd: 97,22, **adam: 97,30**, rmsprop: 96,78	adam
Швидкість Adam	1e-3: 97,30, **1e-4: 97,52**, 1e-5: 95,78, 1e-6: 81,07	1e-4
Епох	5: 96,93, **10: 97,52**, 15: 97,22, 20: 97,26, 25: 97,19	10

Для ResNet50 найкращим виявився найменший класифікатор — **Dense 32** (97,48%): ознаки ResNet50 після GAP уже добре розділяють котів і собак, а більший шар швидше перенавчається (train 99,8% при Dense 1024, val нижча). Dropout 0,25–0,6 дає майже однаковий результат, 0,8 — забагато для 32 нейронів (train падає до 93%). Додатковий Dense, як і для інших мереж, не допомагає. Великі міні-вибірки (50–125) трохи гірші. Серед оптимізаторів найкращий Adam, RMSprop знову має найбільші втрати; швидкості 1e-5 і 1e-6 замалі для 10 епох. Обрано: **Dense 32, Dropout 0,6, batch 32, Adam 1e-4, 10 епох** — лише 65 601 параметр навчається; на тесті **97,78%** (лаба 8: 97,33%).

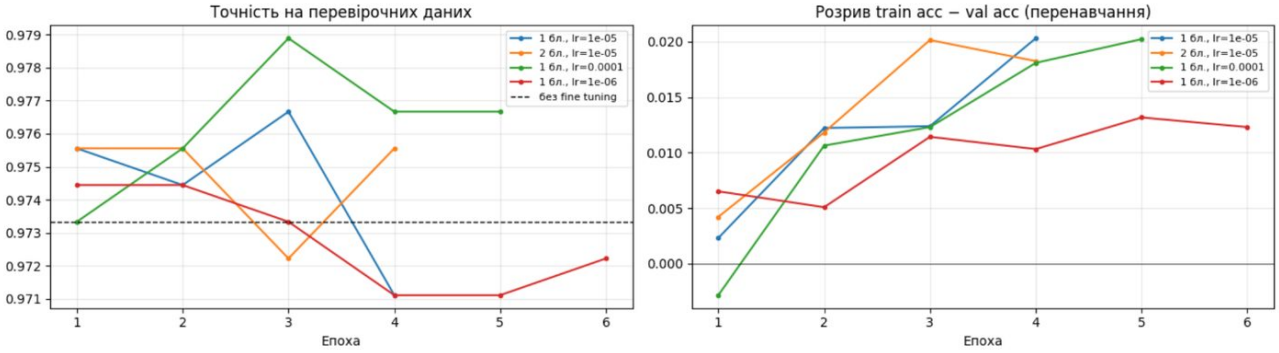


ResNet50: перенесення навчання з підібраними гіперпараметрами — test acc 97,78% (лаба 8: 97,33%)

Тонке налаштування

Розморожено 1 блок (conv5_block3) або 2 блоки (conv5_block2 + 3); BatchNormalization заморожені; Adam з різною швидкістю, до 10 епох з EarlyStopping за val loss (patience 3).

Блоків	lr	Навчаються параметрів	Епох (найкр.)	Train acc	Val acc	Val loss	Test acc, %	Test loss	Час, с
1	1e-5	4 525 121	4 (1)	97,79	97,56	0,0644	97,67	0,0661	82
2	1e-5	8 984 641	4 (1)	97,98	97,56	0,0662	**97,78**	0,0714	83
1	1e-4	4 525 121	5 (2)	98,62	97,56	0,0601	97,44	0,0823	92
1	1e-6	4 525 121	6 (3)	98,48	97,33	0,0664	97,67	0,0679	108



ResNet50: точність на перевірці і розрив train − val для варіантів fine tuning (пунктир — без fine tuning)

Тонке налаштування дало 97,44–97,78% — на рівні перенесення навчання (97,78%), але не вище. Найкраща епоха — 1–3: розморожені блоки швидко підлаштовуються під навчальні фото (розрив train − val зростає до 2%), і EarlyStopping зупиняє навчання на 4–6 епосі. Варіант з lr 1e-4 на 3-й епосі дав найвищу точність на перевірці (97,9%), але на тесті — найнижчу (97,44%).

6. Порівняння трьох моделей і з лабораторною 8

Мережа	Лаба 8 (без підбору)	Лаба 9: перенесення + підбір	Лаба 9: найкращий fine tuning	Найкраща модель	Власні фото	Тварини в траві
VGG16	97,56	95,44	96,22	fine tuning, 1 блок, lr=1e-5	19/20 (19)	5/6 (5)
InceptionV3	96,89	98,00	98,00	перенесення навчання (fine tuning не перевищив)	19/20 (19)	6/6 (6)
ResNet50	97,33	97,78	97,78	перенесення навчання (fine tuning не перевищив)	19/20 (19)	6/6 (6)

Табл. Точність на тестовому наборі, %; у дужках — результат лаби 8

Найкраща модель лабораторної — **InceptionV3: 98,00%** на тестовому наборі. Підбір гіперпараметрів покращив InceptionV3 (+1,1% до лаби 8) і ResNet50 (+0,45%), а для VGG16 результат на тесті знизився (97,56% → 95,44%): у VGG16 після Flatten класифікатор великий (4,2 млн параметрів) і дуже впевнений у відповідях (test loss 0,62), тому вибір за перевірочними даними з різницею в 3–5 фото виявився невдалим для тесту. **Тонке налаштування** допомогло лише VGG16 (+0,8% до моделі з підібраними гіперпараметрами). Для InceptionV3 і ResNet50 воно не дало приросту: на 4 200 навчальних

фото розморожені блоки (4,5–12 млн параметрів) за 1–3 епохи підлаштовуються під навчальний набір і починають перенавчатися, а ознаки ImageNet і так добре підходять — у ImageNet є понад 100 порід собак і кілька видів котів. Різниця між варіантами 0,1–0,5% — це 1–5 фото з 900, тобто в межах випадкового розкиду. Порівняно з власною CNN з лаби 7 (84,75%) усі моделі помиляються в 6–7 разів рідше.

7. Розпізнавання нових зображень

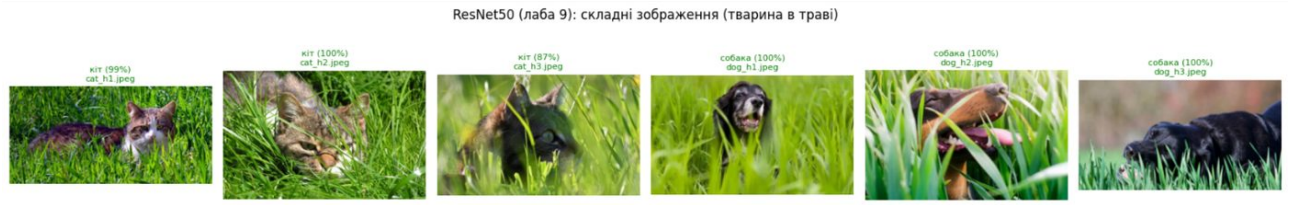
Ті самі 20 власних фото і 6 «складних» (тварина у високій траві), що в лабах 7–8.



VGG16 (лаба 9): тварини в траві — 5 з 6



InceptionV3 (лаба 9): тварини в траві — 6 з 6



ResNet50 (лаба 9): тварини в траві — 6 з 6

Усі три моделі, як і в лабі 8, розпізнали **19 з 20** звичайних фото — помилка на тому самому cat_3 (маленький кіт на камінні біля моря). На складних фото InceptionV3 і ResNet50 — **6 з 6**, VGG16 — 5 з 6: тепер правильно розпізнано cat_h1 (у лабі 8 — помилка), але помилка на темному коті cat_h3 (собака 89%). Отже, на власних фото після підбору гіперпараметрів і тонкого налаштування розпізнавання **не погіршилось і залишилось на максимальному рівні** (19/20, 6/6), а порівняно з власною CNN з лаби 7 (16/20 і 3/6) — суттєво краще. Помітно покращити вже нема куди: єдине «важке» фото cat_3 потребує або обрізки, або навчання на більших зображеннях.

8. Збережені моделі

Найкращі моделі збережено у форматі `.keras` і завантажено з Colab разом із CSV-таблицями всіх експериментів (`_hyperparams.csv`, `_finetune.csv`, `_own.csv`, `_summary.csv`):

- `lab9_VGG16_best.keras`, `lab9_InceptionV3_best.keras`, `lab9_ResNet50_best.keras` — для порівняння в наступній лабораторній.

Висновки

- Тонке налаштування — це донавчання останніх згорткових блоків попередньо навченої мережі з дуже малою швидкістю навчання після того, як класифікатор уже навчено. Розморожувати варто 1 блок; 2 блоки і швидкість 1e-4 швидко призводять до перенавчання або «ламають» ваги ImageNet, 1e-6 — майже нічого не змінює.
- Підбір гіперпараметрів: найбільше впливає швидкість навчання (Adam 1e-4 — найкраще для всіх мереж; 1e-6 — недонавчання); додатковий Dense ніде не покращив; оптимальна кількість нейронів залежить від мережі (VGG16 і InceptionV3 — 512, ResNet50 — 32); 10–15 епох достатньо.
- Обчислення ознак замороженої мережі один раз дозволило провести 30 експериментів на мережу за ~4 хв замість ~1 год.
- Найкращі результати: InceptionV3 — 98,00%, ResNet50 — 97,78%, VGG16 — 96,22% (з fine tuning). Для InceptionV3 і ResNet50 тонке налаштування не перевищило перенесення навчання з підібраними гіперпараметрами — при малому наборі даних (4 200 фото) заморожених ознак ImageNet достатньо.
- На власних фото — 19 з 20 і 5–6 з 6 у траві, як і після лаби 8; значно краще, ніж власна CNN з лаби 7. Найкращі моделі збережено для наступної лабораторної.